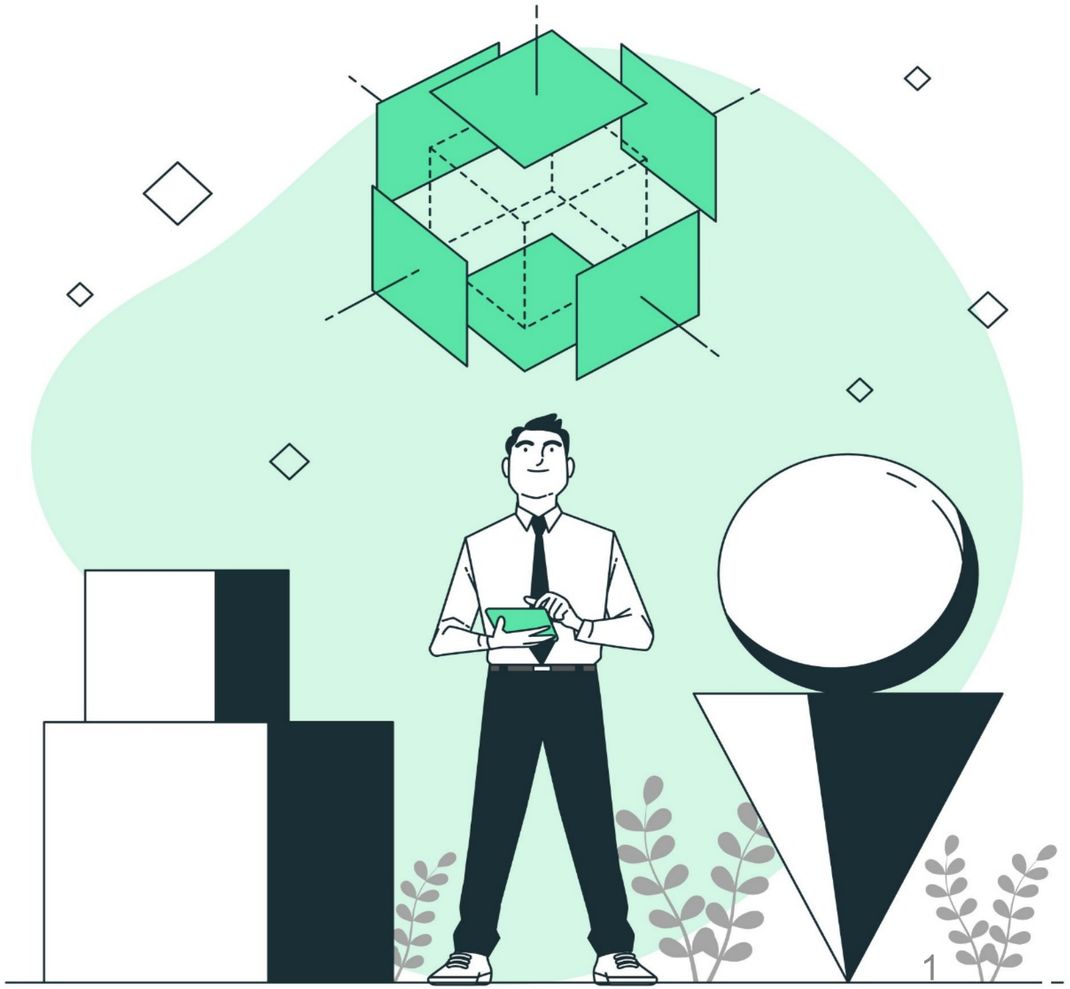
La Programmation orientée objet

Avec le langage C++

Cours 2:
Classes et objets

Dr. Nesrine Zaghdoud



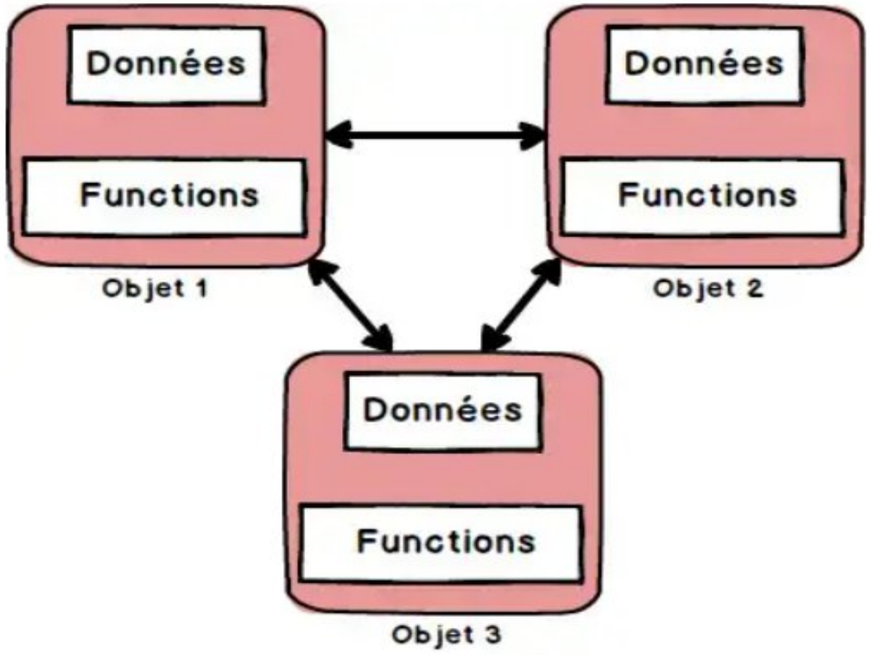
PLAN

1. Introduction
2. Notion d’objet
3. Notion de classe
4. L’autoréférence
5. L’encapsulation
6. Les modificateurs des membres
7. Les méthodes « get » et « set »
8. Notion de constructeur
9. Notion de destructeur

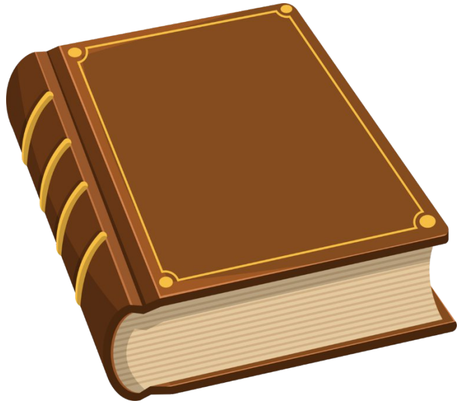
1. Introduction

Rappel

- La programmation orientée objet est un concept de programmation qui se concentre sur l’objet.



Les objets (Tangible – intangible)



Les objets dans un programme de gestion d’une université

L’environnement de l’université

Etudiant

Cours

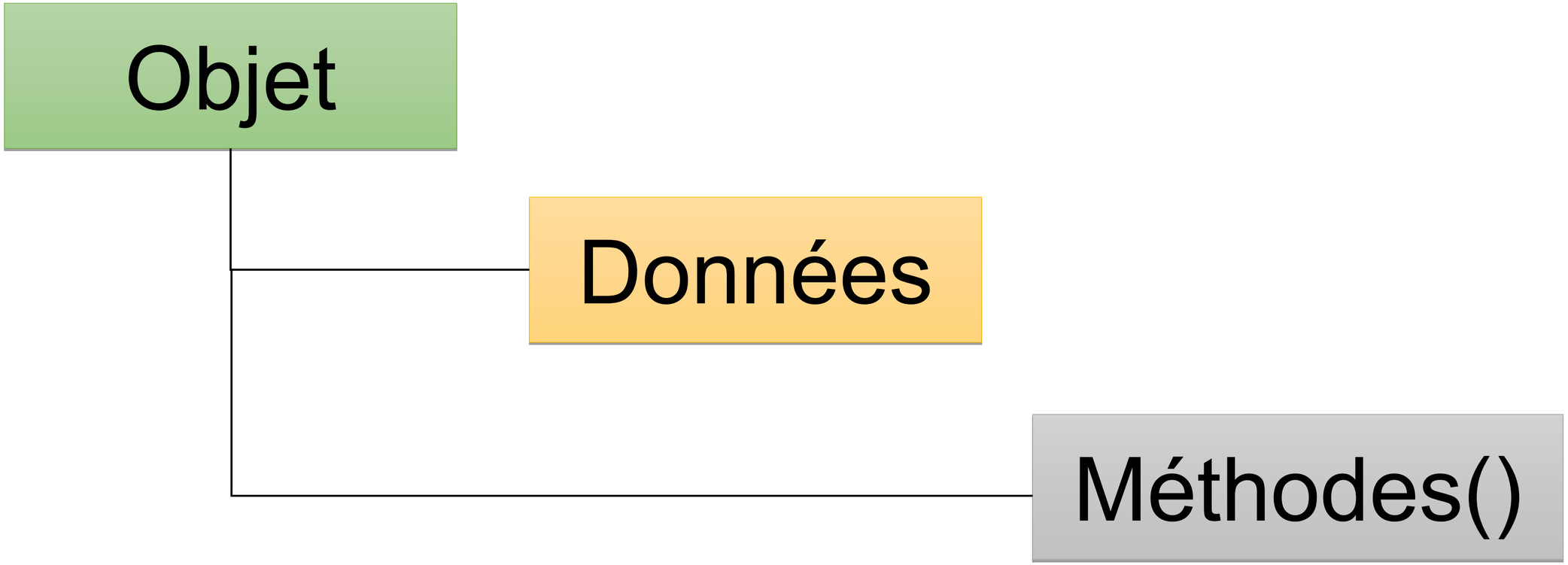
Bâtiment

Enseignant

Salle

Administratif

De quoi est composé l’objet?



De quoi est composé l’objet?

Etudiant

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()

…

De quoi est composé l’objet?

Voiture

Données

1. Usine
2. modèle
3. Capacité en carburant
4. Couleur

…

Méthodes()

1. Set_nomUsine()
2. Set_modèle()
3. Changer_couleur()
4. Get_Voiture_info()

…

Pourquoi on est besoin de classe?

Etudiant 1

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()

Etudiant 2

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()

Etudiant 3

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()

Pourquoi on est besoin de classe?

Classe Etudiant

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()

Etudiant 1

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()

Etudiant 2

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()

Etudiant 3

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()

Pourquoi on est besoin de classe?

Classe Etudiant

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()

Email

Etudiant 1

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()

Etudiant 2

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()

Etudiant 3

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()

Pourquoi on est besoin de classe?

Classe Etudiant

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe
6. **Email**

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()

Etudiant 1

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe
6. **Email**

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()

Etudiant 2

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe
6. **Email**

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()

Etudiant 3

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe
6. **Email**

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()

Pourquoi on est besoin de classe?

Classe Etudiant

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe
6. Email

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()

Afficher_info_Etud()

Etudiant 1

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe
6. Email

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()

Etudiant 2

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe
6. Email

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()

Etudiant 3

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe
6. Email

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()

Pourquoi on est besoin de classe?

Classe Etudiant

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe
6. Email

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()
5. **Afficher_info_Etud()**

Etudiant 1

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe
6. Email

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()
5. **Afficher_info_Etud()**

Etudiant 2

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe
6. Email

Méthodes()

1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()
5. **Afficher_info_Etud()**

Etudiant 3

Données

1. Num_étudiant
2. Nom
3. Date_Naiss
4. MoyG
5. Id_Classe
6. Email

Méthodes()

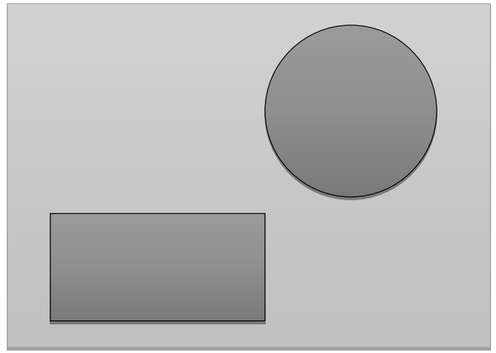
1. Get_NumEtud()
2. Get_Nom()
3. Modifier_MoyG()
4. Changer_id_Classe()
5. **Afficher_info_Etud()**

Objet et classe

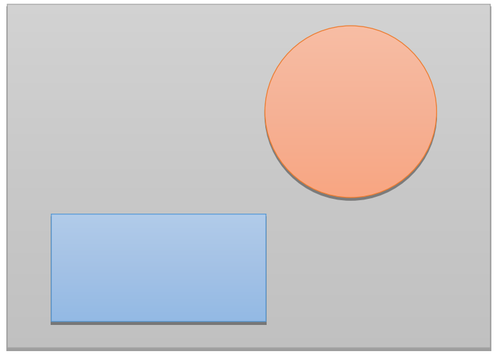
- Une **classe** n’est rien d’autre que **la description d’un ensemble d’objets** ayant **une structure de données commune** et disposant **des mêmes méthodes.**
- Chaque classe définit **un nouveau type de données** qui permettra de créer (instancier) des objets de ce type (on dit aussi qu’un objet est une « instance » de sa classe).
- L’objet est une association des données et des procédures (qu’on appelle alors méthodes) agissant sur ces données.

Objet et classe

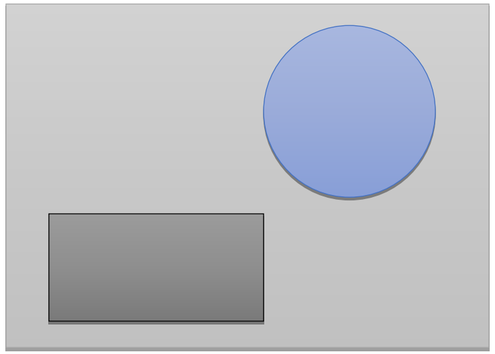
- Classe



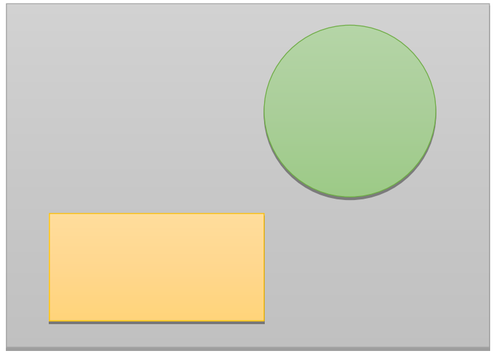
- Instance
  - Objet 1



  - Objet 2



  - Objet 3



Objet et classe

- Une classe est constituée de :
  - **Variables**, appelées ***données***, ***attributs*** ou variables membres,
  - **Fonctions**, appelées ***méthodes*** ou fonctions membres.
    - **Constructeurs**: méthodes servant à créer des objets.

Syntaxe d'écriture d'une classe

- La syntaxe générale pour déclarer une classe:

```
class Nom_de_la_classe{

        Déclaration des attributs;
        Déclarations de méthodes;

        Déclaration de constructeurs;

        Déclarations de destructeurs;

}; //N'oubliez pas le point virgule à la fin
```

Déclaration des attributs et des méthodes

- Les attributs et les méthodes sont aussi nommés les membres de la classe:
  - Les attributs sont les membres données (data members)
  - Les méthodes sont les membres fonctions (function members)

Créer votre premier classe ☺

```
class Rectangle
{
        double hauteur;
        double largeur;

    void setHauteur(double h){ hauteur=h;}
    void setLargeur(double l){ largeur=l;}
    double surface(){ return hauteur*largeur;}
    double perimetre(){
       return 2*(hauteur+largeur); }
};
```

Nom de la classe

Attributs

Méthodes

Création d'un objet

```
int main()
{
    Rectangle r1;
    r1.setHauteur(5.5);
    r1.setLargeur(8.0);
    cout << "La surface du rectangle est " << r1.surface() << "\n";
    cout << "Le perimetre du rectangle est " << r1.perimetre();
    return 0;
}
```

Création de l'objet r1

Définition de la hauteur et la largeur

Affichage de la surface et le périmètre dy rectangle

Note

- Lors de l’exécution de votre programme, vous ne pouvez pas modifier votre objet depuis le programme principal puisque tout les membres de la classe sont par défaut **privé**.

```
C:\Users\ms...        In function 'int main()':
C:\Users\ms...   23   error: 'void Point::initialiser(float, float)' is private within this context
```

- Vous devez ajouter la propriété **public** avant les méthodes pour que vous pouvez les utiliser dans votre programme principal.

Créer votre premier classe ☺

```
class Rectangle
{
        double hauteur;
        double largeur;
    public:
    void setHauteur(double h){ hauteur=h;}
    void setLargeur(double l){ largeur=l;}
    double surface(){ return hauteur*largeur;}
    double perimetre(){ return 2*(hauteur+largeur); }
};
```

Créer votre premier classe ☺

```
class Rectangle
{
    private:
        double hauteur;
        double largeur;
    public:
    void setHauteur(double h){ hauteur=h;}
    void setLargeur(double l){ largeur=l;}
    double surface(){ return hauteur*largeur;}
    double perimetre(){ return 2*(hauteur+largeur); }
};
```

Un autre exemple ☺

```
class Point
{
  private:
   float x;
   float y;
  public:
   void initialiser (float abs, float ord){ x=abs; y=ord;}
   void translater(float dx, float dy){
   x+=dx; y+=dy; }
   void affiche(){
   cout << "Les coordonnees du point sont " << x << " " << y << "\n" ;
   }
};
```

Nom de la classe

Attributs

Méthodes

Création d'un objet

```
int main()
{
    Point p1;
    p1.initialiser(2.5, 7.5);
    p1.affiche();
    p1.translater(2,2);
    p1.affiche();
    return 0;
}
```

Création de l'objet p1

Ajouter les coordonnées du point.

Afficher les coordonnées du point.

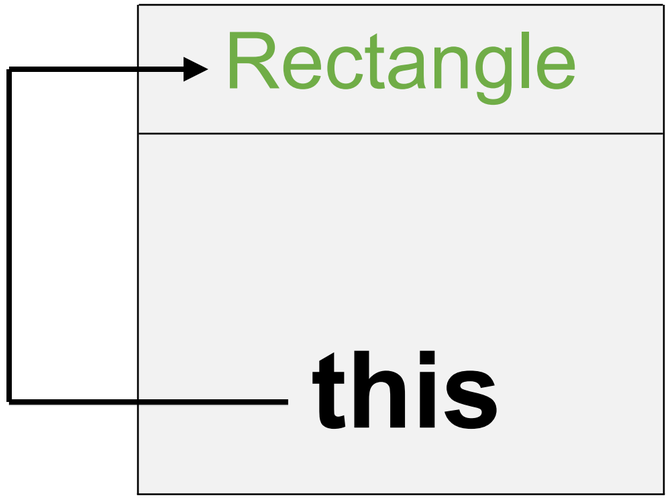
Translater le point.

Afficher les coordonnées du point.

L’autoréférence: Le pointeur « this »

- Dans toutes les classes, on dispose d'un pointeur ayant pour nom **this**, qui pointe vers l'objet actuel.

Chaque objet (ici de class Rectangle) possède un pointeur **this** qui pointe vers l'objet lui-même!



L'autoréférence: Le pointeur « this »

- Ce type de notation artificielle peut s'avérer pratique dans l'écriture de certains constructeurs. Par exemple, le constructeur suivant :

```
Rectangle(double h, double l)
{
hauteur=h;
largeur=l;
}
```

Correct

```
Rectangle(double hauteur, double largeur)
{
this->hauteur=hauteur;
this->largeur=largeur;
}
```

Correct

```
Rectangle(double hauteur, double largeur)
{
hauteur=hauteur;
largeur=largeur;
}
```

Ambiguë, génère une erreur lors de la compilation

L'encapsulation

- L'encapsulation est un mécanisme qui consiste à **rassembler les données** et **les méthodes** au sein d'une classe **en cachant l'implémentation de l'objet**.
- Le principe d'encapsulation permet aux objets de se représenter sous deux vues possibles:
  - **La vue externe:** Elle définit les attributs et les méthodes accessibles aux utilisateurs de l'objet et ne fournit aucun accès aux définition internes de l'objet (en la mettent privée) .
  - **La vue interne:** Elle donne tous les détails sur la constitution interne de l'objet.

Il est conseillé de mettre tous **les attributs d'une classe privés** pour s'en profiter des avantages de l'encapsulation.

Tout ce qui rend la POO puissante vient de là.

L’encapsulation: Exemple 1

- **Gestion des données privées**
- Imaginons une classe **CompteBancaire** avec des attributs privés comme le solde et des méthodes pour déposer et retirer de l'argent. L'encapsulation protège l'intégrité des données et empêche les modifications non autorisées

L’encapsulation: Exemple 2

- **Validation des entrées**

L'encapsulation permet également de valider les entrées avant de modifier les données internes. Par exemple, dans une classe **Personne**, on peut s'assurer que l'âge est toujours un nombre positif avant de modifier l’attribut âge.

Avantages de l'encapsulation

- **Sécurité des données** : L’encapsulation permet d’interdire l’accès direct aux détails internes de l’objet et favorise l’intégrité des données.
- **Modularité** : Les objets encapsulés peuvent être réutilisés dans différents programmes.
- **Facilité de maintenance** : Le code encapsulé est plus facile à tester et à modifier sans affecter d’autres parties de programme.
- …

Les modificateurs des membres

Les modificateurs déterminent l'accessibilité et la visibilité des membres d'une classe (attributs et méthodes):

1. **Public:** Les membres déclarés comme public sont accessibles depuis n'importe où dans le programme.

```
class Exemple {
    public:
    int publicMembre; // Accessible de partout
};
```

Les modificateurs des membres

2. **Private:** Les membres déclarés comme private sont accessibles uniquement au sein de la classe dans laquelle ils sont déclarés. Ils ne peuvent pas être accédés directement depuis les objets de cette classe ou les sous-classes.

```
class Exemple {
    private:
    int privateMembre;
    // Accessible uniquement au sein de la classe Exemple
};
```

Les modificateurs des membres

3. **Protected :** Les membres déclarés comme protected sont accessibles au sein de la classe dans laquelle ils sont déclarés, ainsi que dans les classes dérivées (sous-classes).

```
class Exemple {
    protected:
    int protectedMembre;
    // Accessible dans Exemple et ses sous-classes
};
```

Les modificateurs des membres

- Par défaut, si aucun modificateur d'accès n'est spécifié, les membres sont considérés comme **private**.

```
class Exemple {
    int defaultMembre;
    // Par défaut, c'est private
};
```

On résume ☺

- Pour définir une classe:

N'oubliez pas d'ajouter les **modificateurs** appropriés pour les attributs et les méthodes

```
class Nom_de_la_classe{

        Type nom_attribut1;
        Type nom_attribut2;
        ⋮
        Type_retour nom_methode1(type param1, type param2,…);

        Type_retour nom_methode2(type param1, type param2,…);
        ⋮

}; //N'oubliez pas le point virgule à la fin
```

On résume ☺

- Pour définir une classe:

Par exemple ici, on définit les attributs comme privés et les méthodes comme publiques.

```
class Nom_de_la_classe{
    private:
        Type nom_attribut1;
        Type nom_attribut2;
    public:
        Type_retour nom_methode1(type param1, type param2,…);

        Type_retour nom_methode2(type param1, type param2,…);
        ⋮

}; //N'oubliez pas le point virgule à la fin
```

Les Méthodes «get» et «set»

- Si on a besoin d’utiliser des attributs privés depuis l’extérieur de la classe.
- On utilise des méthodes publiques comme:
  - Manipulateurs des attributs (« méthodes set ») :
  - Accesseurs des attributs (« méthodes get ») :

Les Méthodes «get» et «set»

- Manipulateurs (« méthodes set ») :
  - Modification, Affectation de l'argument à une variable d'instance précise.

```
void setHauteur(double H){hauteur=H;}
void setLargeur(double L){largeur=L;}
```

- Accesseurs (« méthodes get ») :
  - Consultation, Retour de la valeur d'une variable d'instance précise.

```
double getHauteur() {return hauteur;}
double getLargeur() {return largeur;}
```

Constructeur et destructeur

- Le **constructeur** est une méthode **appelée automatiquement** à chaque fois que **l’on crée un objet** basé sur cette classe.
- Le **destructeur** est une méthode **appelée automatiquement** lorsqu’un **objet est détruit**, par exemple à la fin de la fonction dans laquelle il a été déclaré ou utilisé.

Le constructeur

- Comme son nom l’indique, c’est une méthode qui sert à construire l’objet. Elle est automatiquement appelée dès qu’on crée un objet.

```
Rectangle r1, r2;
```

```
Point p1;
```

Le rôle du constructeur

- **Le rôle principal** du constructeur est **d'initialiser les attributs**.
  - ❖ Dans les exemples qu'on a utilisés jusqu'à maintenant on a créé des objets mais ils ne sont pas initialisés lors de leurs créations.
  - ❖ On a utilisé un **constructeur par défaut** qui est automatiquement créé par le compilateur.
  - ❖ Il crée un objet vide, ce qui peut engendrer des problèmes dans certains cas.
- On a très souvent besoin de **créer soi-même un constructeur** qui remplace ce constructeur vide par défaut,
  - ❖ ce qui permet de s'assurer que **nos objets seront initialisés** lors de leurs création.

Créer un constructeur

- Pour créer un constructeur, il y a deux règles à respecter:
  - Il faut que le constructeur ait **le même nom que la classe** donc s’appeler Rectangle, Point par exemple.
  - La méthode **ne doit rien renvoyer**, pas même void.

Le constructeur est une **méthode**

sans aucun type de retour.

Créer un constructeur

- 1ère définition:

```
class Point {
    private:
        int x;
        int y;
    public:
       Point() {        // Constructeur par défaut
           x = 5;  y = 5;
        }
        void afficher() {   // Méthode pour afficher les coordonnées du point
           cout << "Point(" << x << ", " << y << ")" << endl;
        }
};
```

Surcharger le constructeur

```
int main() {
    Point p1;           // Appelle le constructeur
    p1.afficher();      // Affiche Point(5, 5)

   return 0;
}
```

Surcharger le constructeur

- **La notion de surchargement** du constructeur en programmation orientée objet permet de **créer plusieurs versions d'un constructeur** au sein d'une même classe.
- Chaque version diffère **par le nombre et/ou le type des paramètres** qu'elle prend en entrée.
- Cela permet de fournir différentes façons d'initialiser un objet selon les informations disponibles.

Surcharger le constructeur

```
class Point {
    private:
        int x;
        int y;
    public:
    Point() {// Constructeur par défaut
            x = 0;  y = 0; }
    Point(int a) {// Constructeur avec un paramètre (coordonnée x)
            x = a; y = 0; }
    Point(int a, int b) {// Constructeur avec deux paramètres (coordonnées x et y)
            x = a;  y = b; }
    void afficher() {// Méthode pour afficher les coordonnées du point
            cout << "Point(" << x << ", " << y << ")" << endl;
        }
};
```

Surcharger le constructeur

```
int main() {
    Point p1;          // Appelle le constructeur par défaut
    Point p2(5);        // Appelle le constructeur avec un paramètre
    Point p3(3, 4);     // Appelle le constructeur avec deux paramètres

    p1.afficher();      // Affiche Point(0, 0)
    p2.afficher();      // Affiche Point(5, 0)
    p3.afficher();      // Affiche Point(3, 4)

   return 0;
}
```

Le constructeur de copie

- Le constructeur de copie en programmation orientée objet est un constructeur spécial utilisé pour créer un nouvel objet en copiant les membres d'un objet existant.
- Il est particulièrement utile pour créer une copie exacte d'un objet tout en garantissant que chaque objet reste indépendant.

```
ClassName( const ClassName &p);
```

Le constructeur de copie

```
class Point {
private:
    int x;
    int y;
public:
   Point() {x = 0;  y = 0; }
   Point(int a) {x = a; y = 0; }
   Point(int a, int b) {x = a;  y = b; }
   Point(const Point& p) {   // Constructeur de copie
        x(p.x);
        y(p.y);
        cout << "Constructeur de copie appelé." << endl;}

    // Méthode pour afficher les coordonnées du point
    void afficher() {
        cout << "Point(" << x << ", " << y << ")" << endl;
    }
};
```

Le constructeur de copie

```
int main() {

Point p1(3, 4); // Appelle le constructeur avec deux paramètres

Point p2(p1); // Appelle le constructeur de copie

p1.afficher(); //Affiche Point(3, 4)

p2.afficher(); // Affiche Point(3, 4)

return 0; }
```

Le destructeur

- Le destructeur est une méthode spéciale qui est automatiquement appelée lorsqu'un objet est détruit ou sort de portée.
- Le rôle principal du destructeur est de libérer les ressources allouées par l'objet pendant sa durée de vie, telles que la mémoire dynamique ou les descripteurs de fichiers.

Le destructeur

```
class Point {
private:
    int x;
    int y;
public:
   Point() {x = 0;  y = 0; }
   Point(int a) {x = a; y = 0; }
   Point(int a, int b) {x = a;  y = b; }
    ~Point() {// Destructeur
        cout << "Destructeur appele pour Point(" << x << ", " << y << ")" << endl;
     }
    void afficher() {
        cout << "Point(" << x << ", " << y << ")" << endl;}
};
int main() {
    Point p1(3, 4); // Allocation dynamique d'un objet Point
    p1.afficher();
    return 0;
}
```

```
Point(3, 4)
Destructeur appele pour Point(3, 4)

Process returned 0 (0x0)   execution time : 1.016 s
Press any key to continue.
```